HTML & CSS

Interview Cheat Sheet

A concise, accurate reference for front-end interview prep

1. HTML Basics

1.1 What is HTML?

- HTML (HyperText Markup Language) is the standard markup language used to define the structure and content of web pages.
- It is not a programming language — it has no logic or control flow — it describes structure using elements.
- Browsers parse HTML into the DOM (Document Object Model), which CSS styles and JavaScript manipulates.

1.2 Basic HTML Document Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>

</body>
</html>
```

Note: the viewport meta tag is included above because it's essential for responsive design and commonly expected in interview answers, even though it's often left out of quick examples.

1.3 Headings

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
...
<h6>Heading 6</h6>
```

There are six heading levels, h1–h6. h1 should be used once per page as the main heading for SEO and accessibility; heading levels should not be skipped.

1.4 Paragraph

```
<p>This is a paragraph.</p>
```

1.5 Tags vs. Elements

```
<h1>Hello</h1>
```

Term	Definition
Opening tag	<h1>
Closing tag	</h1>

Element	The complete unit: Hello (opening tag + content + closing tag)

1.6 Void (Self-Closing) Elements

Void elements have no closing tag and cannot contain content. Common examples:

-
 — line break
- <hr> — horizontal rule
- <img> — image
- Also commonly asked: <input>, <meta>, <link>

1.7 Lists

Ordered List

```
<ol>
  <li>Item</li>
</ol>
```

Unordered List

```
<ul>
  <li>Item</li>
</ul>
```

Definition List

```
<dl>
  <dt>HTML</dt>
  <dd>Markup Language</dd>
</dl>
```

1.8 Anchor Tag

```
<a href="https://google.com" target="_blank" rel="noopener noreferrer">Google</a>
```

target="_blank" opens the link in a new tab. When using target="_blank", it's best practice to add rel="noopener noreferrer" to prevent the new page from accessing window.opener (a security/performance concern).

1.9 Images

```
<img src="image.jpg" alt="Description">
```

- Absolute path — full URL (e.g., https://site.com/image.jpg)
- Relative path — relative to the current file (e.g., ./images/photo.jpg)
- alt improves accessibility (used by screen readers) and displays if the image fails to load.

1.10 div vs. span

Property	div	span
Display type	Block-level	Inline-level
Behavior	Starts on a new line; takes full available width	Flows within a line; only as wide as its content
Typical use	Grouping larger sections/layout blocks	Styling a small piece of text inline

2. HTML Forms & Media

2.1 Basic Form

```
<form action="/submit" method="POST">
    <label for="name">Name</label>
    <input type="text" id="name" name="name">

    <label for="email">Email</label>
    <input type="email" id="email" name="email">

    <label for="pwd">Password</label>
    <input type="password" id="pwd" name="pwd">

    <input type="submit" value="Submit">
</form>
```

Note: pairing each <label> with its <input> via a matching for/id (or by wrapping the input inside the label) is a common accessibility interview question.

2.2 GET vs. POST

Property	GET	POST
Data location	Appended to the URL as a query string	Sent in the request body
Visibility / security	Visible in URL, browser history, logs — less secure	Not visible in the URL — more secure for sensitive data
Data limit	Limited by URL length (browser/server dependent)	No practical size limit
Typical use	Fetching/searching data (idempotent, cacheable)	Submitting forms, login, creating/updating resources
Bookmarkable / cacheable	Yes	No

2.3 Select Dropdown

```
<select name="cars">
    <option value="volvo">Volvo</option>
    <option value="bmw">BMW</option>
</select>
```

2.4 Audio

```
<audio controls>
    <source src="song.mp3" type="audio/mpeg">
</audio>
```

2.5 Video

```
<video controls>
    <source src="video.mp4" type="video/mp4">
</video>
```

2.6 iframe

```
<iframe src="https://example.com" title="Embedded page"></iframe>
```

Used to embed another HTML document/webpage within the current page. A title attribute is recommended for accessibility.

2.7 Tables

```
<table>
    <thead>
        <tr><th>Header</th></tr>
    </thead>
    <tbody>
        <tr><td>Data</td></tr>
    </tbody>
    <tfoot>
        <tr><td>Footer</td></tr>
    </tfoot>
</table>
```

Tag	Purpose
<table>	Defines the table
<thead>	Groups header content
<tbody>	Groups the main body content
<tfoot>	Groups footer content
<tr>	Table row
<th>	Header cell (bold, centered by default)
<td>	Standard data cell

3. CSS Basics

3.1 What is CSS?

CSS stands for Cascading Style Sheets. It controls the presentation — layout, color, spacing, typography — of HTML elements.

```
selector {
    property: value;
}

/* Example */
h1 {
    color: red;
}
```

3.2 CSS Selectors

Selector Type	Syntax	Description
Element selector	h1 { } p { }	Targets all elements of that type
Class selector	.title { }	Targets all elements with class="title"
ID selector	#title { }	Targets the single element with id="title" (should be unique per page)
Attribute selector	input[type="text"] { }	Targets elements matching an attribute/value
Group selector	h1, p, div { }	Applies the same rule to multiple selectors
Descendant selector	div p { }	Targets any <p> nested anywhere inside a
Child selector	div > p { }	Targets only direct <p> children of a
Chaining (compound) selector	h1.title { }	Targets an <h1> that also has class="title"
Pseudo-class	a:hover { }	Targets an element in a particular state (hover, focus, etc.)

3.3 Colors

```
color: red;              /* named color */
color: #FF0000;          /* hex */
color: rgb(255, 0, 0);   /* rgb */
background-color: blue;
```

3.4 Fonts

```
font-family: Arial, sans-serif;
font-size: 16px;
font-weight: bold;
```

Common length units:

- px — fixed pixel value
- em — relative to the font size of the parent element
- rem — relative to the root (<html>) font size, more predictable than em
- % — relative to the parent element's corresponding property

3.5 Text Alignment

```
text-align: left;
text-align: center;
text-align: right;
```

4. Box Model, Display & Position

4.1 Box Model

Every element is a box made up of four layers, from the inside out:

- Content — the actual text/image
- Padding — space between content and border
- Border — the edge surrounding the padding
- Margin — space outside the border, separating it from other elements

```
margin: 20px;
padding: 10px;
border: 2px solid black;
```

box-sizing: border-box (commonly asked follow-up) makes width/height include padding and border, instead of the default content-box behavior.

4.2 Display Property

Value	Behavior	Examples
block	Starts on a new line; takes full available width; width/height respected	div, p, h1
inline	Flows within the line; width/height are ignored; only left/right margin respected	span, a, img*
inline-block	Flows inline but respects width & height	—
none	Removed from layout entirely (not	display: none;

Value	Behavior	Examples
	just hidden visually)	

*img is technically inline but is a replaced element, so it does respect width/height — a nuance worth mentioning in interviews.

4.3 Position

Value	Behavior
static	Default. Element follows normal document flow; top/left/etc. have no effect.
relative	Positioned relative to its own normal position, using top/left/right/bottom.
absolute	Positioned relative to the nearest ancestor with position other than static (its 'positioned' ancestor); removed from normal flow.
fixed	Positioned relative to the browser viewport; stays fixed even when scrolling.
sticky	Behaves like relative until a scroll threshold, then behaves like fixed within its container.

z-index controls stacking order for positioned elements (position other than static) — higher values render on top.

4.4 Float

```
float: left;
float: right;
```

Used in older, pre-Flexbox/Grid layouts to wrap text around elements or build columns. Largely superseded by Flexbox and Grid for layout purposes today, though still used for text-wrap-around-image effects.

5. Flexbox (One-Dimensional Layout)

```
display: flex;
```

5.1 Main Axis Direction

```
flex-direction: row;        /* default, left to right */
flex-direction: column;     /* top to bottom */
```

5.2 Wrapping

```
flex-wrap: nowrap;       /* default */
flex-wrap: wrap;
flex-wrap: wrap-reverse;
```

5.3 Justify Content (Main Axis)

```
justify-content: flex-start;
justify-content: flex-end;
justify-content: center;
justify-content: space-between;
justify-content: space-around;
justify-content: space-evenly;
```

5.4 Align Items (Cross Axis)

```
align-items: flex-start;
align-items: flex-end;
align-items: center;
align-items: stretch;   /* default */
```

5.5 Align Self

```
align-self: flex-start;
```

Overrides align-items for a single flex item.

5.6 Gap

```
gap: 20px;
```

5.7 Order

```
order: 2;
```

Changes the visual order of a flex item without changing the underlying HTML/DOM order. Default order is 0.

5.8 Flex Basis, Grow & Shrink

```
flex-basis: 100px;   /* initial main-size before growing/shrinking */
flex-grow: 1;        /* how much the item expands to fill free space */
flex-shrink: 1;      /* how much the item shrinks when space is tight */
```

Shorthand

```
flex: 1 1 100px;   /* grow shrink basis */
```

6. CSS Grid (Two-Dimensional Layout)

```
display: grid;
```

6.1 Columns

```
grid-template-columns: 100px 200px;
grid-template-columns: repeat(2, 200px);
grid-template-columns: 100px auto;
```

6.2 Rows

```
grid-template-rows: 100px 200px;
grid-auto-rows: 300px;
```

6.3 Gap

```
gap: 20px;
```

6.4 Placing Items

```
grid-column: 1 / 3;   /* start line / end line */
grid-row: 2 / 4;
grid-area: 1 / 1 / 3 / 2;  /* row-start / column-start / row-end / column-end */
```

6.5 Flexbox vs. Grid — Quick Distinction

	Flexbox	Grid
Dimensions	One-dimensional (row OR column)	Two-dimensional (rows AND columns together)
Best for	Distributing items along a single axis (navbars, toolbars)	Full page/component layouts with rows and columns

7. Responsive Design, Animations & Transitions

7.1 Media Queries

```
@media (max-width: 600px) {
    /* styles for small screens */
}

@media (min-width: 768px) {
    /* styles for tablets and up */
}

@media (orientation: landscape) {
    /* styles when width > height */
}
```

7.2 CSS Animations

```
@keyframes move {
    0%   { transform: translateX(0); }
    100% { transform: translateX(100px); }
}

div {
    animation-name: move;
    animation-duration: 2s;
}
```

7.3 CSS Transitions

```
transition: all 0.3s ease;
```

Transitions animate a property change smoothly between two states (e.g., on :hover). Animations, by contrast, use @keyframes and can define multiple intermediate steps and run independently of a state change.

8. Common Interview Topics — Quick Reference

- HTML vs. HTML5
- Tags vs. Elements
- Inline vs. Block vs. Inline-block elements
- GET vs. POST
- div vs. span
- Class vs. ID selectors
- CSS Specificity
- Box Model (content-box vs. border-box)
- Display property (block, inline, inline-block, none)
- Position (static, relative, absolute, fixed, sticky)
- Float vs. Flexbox vs. Grid

- Flexbox properties (justify-content, align-items, flex-grow/shrink/basis)
- Grid properties (grid-template-columns/rows, grid-area)
- Media Queries
- CSS Animations vs. Transitions
- Responsive Web Design (RWD)
- Semantic HTML (e.g., <header>, <nav>, <main>, <footer>, <article>, <section>)
- Accessibility (alt, label, ARIA attributes)